TripWise Technical Project and Resume Guide

Agentic AI travel planning and credit card rewards optimization

A technical narrative for resumes, elevator pitches, and engineering interviews. This guide describes the completed target-state product envisioned for the portfolio project.

Prepared for	Himanshu Jain
Project type	Non-commercial student and portfolio project
Version	Completed target-state narrative | October 2026

Recommended positioning: a decision-intelligence platform that combines verified travel evidence, bounded AI assistance, and deterministic rewards optimization.

Project Overview

TripWise is a full-stack travel planning platform that creates personalized itineraries and determines how a traveler should pay for each part of a trip. It combines conversational planning, live and recorded travel evidence, credit card rewards logic, loyalty-point transfers, and human review in one workflow.

The core engineering challenge is that travel planning is ambiguous while financial recommendations must be exact. TripWise separates those responsibilities. Language models interpret preferences, suggest options, critique plans, and explain results. Deterministic Python services calculate every price, reward, fee, cap, discount, and transfer value. The interface displays those computed values without recalculating them.

TECHNICAL SNAPSHOT

Backend tests	Typed Python files	Frontend tests	Browser tests
1,709 passed	**177 clean**	**163 passed**	**458 passed**

System Architecture

Layer	Responsibility
Experience layer	Conversational interview, typed controls, resumable sessions, review and amend flow, editable itinerary, maps, trust badges
Agentic orchestration	Intake, planner, critic, explainer, post-plan intent router, confirmation gates, bounded retries and call budgets
Deterministic kernel	Trip costing, card rewards, offers, fees, transfer paths, cash and points comparison, reproducible recommendations
Evidence gateway	Gondola MCP, Open-Meteo MCP, Tripadvisor and POI sources, FX and airport reference data, fixture replay
Persistence and audit	Accounts, preferences, saved trips, session snapshots, optimistic concurrency, idempotency, provenance and review history

Primary Stack

- Backend: Python, FastAPI, Pydantic, SQLAlchemy, SQLite, Argon2id authentication, and typed domain packages.
- Frontend: Next.js, React, TypeScript, generated OpenAPI clients, Vitest, Playwright, and accessible interaction patterns.
- AI and integrations: bounded LLM workflows, Model Context Protocol adapters, recorded fixtures, strict provider allowlists, provenance, and fail-soft fallbacks.

Agentic AI Design

TripWise uses agentic AI as a controlled workflow rather than a free-roaming chatbot. Each AI capability has a fixed role, a typed input and output, an explicit call budget, and a known place in the orchestration graph. The system can gather information across several turns, maintain session state, propose a plan, review its own proposal, and route later edits without allowing the model to change financial facts or execute irreversible actions.

Workflow component	Bounded responsibility
Intake agent	Converts natural-language trip ideas into structured facts and identifies missing information.
Interview assistant	Suggests bounded adaptive questions and interprets free text while preserving typed answer contracts.
Planner agent	Assembles a candidate itinerary from approved preferences and normalized evidence.
Critic agent	Checks feasibility, preference fit, pacing, and unsupported claims before the plan reaches the user.
Explainer agent	Turns computed artifacts into readable prose without inventing prices, rewards, or transfer values.
Edit router	Classifies post-plan requests and dispatches deterministic edits or scoped replanning.

Control Model

- Every workflow transition uses validated schemas. Invalid or incomplete model output is repaired within a fixed budget or replaced with a deterministic fallback.
- The orchestrator controls tool access, sequencing, retries, and state. Models cannot discover new providers, rewrite prompts, delegate to arbitrary agents, or modify application code at runtime.
- Users approve the confirmed Trip Brief before planning begins. Profile updates, financial-rule changes, transfers, and bookings also require explicit human action.
- Runtime errors remain truthful. Provider or model failures produce typed unavailable or error states rather than fabricated results.

Why This Qualifies as Agentic

The system observes conversational state, determines the next permitted action, invokes approved tools, evaluates the result, and either advances, retries, asks for clarification, or fails safely. Its autonomy is meaningful because it coordinates a multi-step objective across turns and tools. Its boundaries make that autonomy testable and appropriate for a financial decision-support product.

Customer Personality and Preference Model

TripWise begins with a short interactive interview that learns how the customer prefers to travel. The interview captures durable traits separately from trip-specific constraints, so a preference such as a quiet boutique hotel can carry across trips while a fixed budget or travel date remains attached to one plan.

Preference dimension	Examples of captured signals
Travel pace	Relaxed, balanced, or packed days; preferred start and end times; tolerance for transit.
Planning style	Structured versus spontaneous travel, comfort with unplanned time, and appetite for alternatives.
Experience interests	Food, culture, history, nature, nightlife, shopping, wellness, family activities, and adventure.
Comfort profile	Accommodation style, flight preferences, accessibility needs, and tradeoffs between convenience and price.
Financial preferences	Cash budget, points balances, card wallet, willingness to transfer points, and value placed on flexibility.

Interview Mechanics

1. Ask a fixed set of core questions using typed controls such as single select, multi-select, ranges, dates, and ranked preferences.
2. Use a bounded adaptive layer to clarify contradictions or missing high-impact preferences. The interview always terminates within a tested range.
3. Present a structured Trip Brief for review. The user can amend any answer before confirming it.
4. Propose durable profile updates separately. Store them only after the customer approves the change.
5. Resume the session across devices through authenticated server-side persistence and idempotent events.

Personalization Without a Black Box

Preferences influence retrieval filters, candidate ranking, itinerary pacing, budget allocation, and explanation language. The final plan still exposes the evidence and constraints that caused each recommendation. A customer can see whether a suggestion came from a saved preference, a trip answer, a provider observation, or a deterministic financial result.

Deterministic Financial Optimization

The financial kernel answers a practical question: given a trip, a wallet of cards and points, and available travel options, what combination of cash, card rewards, offers, and loyalty transfers produces the best explainable outcome? The kernel performs this work without an LLM and returns a complete calculation trace.

Calculation Rules

- Represent money in integer minor units, percentages in basis points, and point values in fixed-scale integers.
- Apply per-transaction reward floors, category eligibility, spending caps, cap fall-through, annual fees, statement credits, and offer stacking in a specified order.
- Compare cash prices, points requirements, taxes and fees, transfer ratios, opportunity cost, and redemption value on a common deterministic basis.
- Search loyalty-transfer paths while requiring the user to verify availability before transferring points.
- Keep golden scenarios fixed so algorithm changes cannot silently alter expected financial outcomes.

Optimization Output

Artifact	What the user receives
Costed itinerary	Cash cost for each travel and activity line, plus an auditable total.
Payment strategy	Recommended card or cash method for each payable item, including expected rewards and relevant caps.
Offer application	Eligible credits and promotions with deterministic stacking and remaining-value tracking.
Transfer advice	Recommended loyalty transfer path, required balances, residual amounts, and verification steps.
Explanation artifact	Structured facts that the explainer can restate without performing or altering the calculation.

Example Interview Explanation

I designed the system so the LLM never performs money math. The model can explain why a card is recommended, but the amount comes from a deterministic artifact that accounts for category multipliers, floors, caps, fees, and offers. This makes the recommendation reproducible and lets us test every financial edge case independently of model behavior.

MCP Integrations and Evidence Gateway

TripWise isolates external data access behind an application-owned Data Gateway. Provider adapters translate each external response into normalized flight, hotel, weather, place, airport, and foreign-exchange contracts. The planning and financial layers consume those contracts instead of importing provider SDKs directly.

Integration	Role in the platform
Gondola MCP	Read-only flight and hotel discovery through explicit tool allowlists and normalized evidence types.
Open-Meteo MCP	Current conditions, near-term forecasts, historical observations, and clearly labeled climate summaries.
Tripadvisor and POI adapters	Place discovery with quota accounting, transport-derived evidence status, coordinates, and provenance.
Frankfurter	Decimal-safe foreign-exchange reference rates with deterministic micro-unit conversion.
OurAirports and Overture	Open airport and place reference data with license metadata and deterministic offline import.
Fixture replay	Sanitized provider captures used for normal tests so the suite makes no network calls.

Safety Envelope

- Adapters expose only approved read-only tools. Booking, payment, account history, storage, SQL, and mutation tools are denied before dispatch.
- Every request enforces bounds for dates, coordinates, variables, rows, response size, calls per plan, timeout, and retries.
- Kill switches and provider profiles keep live integrations disabled unless the environment explicitly permits them.
- Circuit breakers and typed errors prevent raw provider failures from leaking through the API. The product can fall back to cached or fixture evidence while labeling freshness honestly.
- Every normalized fact carries source, verification time, verification method, confidence, status, and a needs-verification flag.

Why MCP Matters

MCP provides a standard tool protocol, but TripWise does not trust a server merely because it speaks that protocol. The application owns the allowlist, schema validation, authentication scope, budgets, sanitization, normalization, and audit trail. This preserves a stable internal contract when a provider changes and makes each integration replaceable.

Data Integrity Security and Auditability

Financial rules enter the trusted knowledge base through an offline ingestion and review pipeline. Source policies, fetch attempts, snapshots, proposals, decisions, and reverification tasks form an evidence chain. Historical records are insert-only or versioned, and conflicting retries raise typed errors rather than overwriting earlier evidence.

Persistence Guarantees

- Compare-and-swap versioning prevents concurrent reviewers from silently overwriting each other.
- Canonical request fingerprints make repeated client events idempotent while rejecting conflicting reuse of the same id.
- Immutable saved-trip revisions preserve each accepted plan, while user-scoped sessions can resume safely across requests.
- Audit events record every permitted mutable update, preserving the prior value, actor, decision, and timestamp.
- Privacy export and deletion cover account data, preferences, sessions, and saved trips.

Authentication and API

- FastAPI endpoints use Pydantic contracts and a generated OpenAPI client to keep backend and frontend schemas synchronized.
- Passwords use Argon2id. Server-side sessions store hashed tokens, enforce lockout, and revoke prior sessions after password changes.
- Cookie authentication and CSRF-protected mutations reduce exposure of credentials in browser code.
- Request-time packages remain separated from ingestion and provider implementations through structural import checks.

Human Approval Boundaries

Agents can propose a preference update, an itinerary, or an interpretation of a financial source. A person approves durable preference changes and financial facts. The traveler confirms the brief before planning and verifies award availability before transferring points. The system never executes a booking or points transfer on the user's behalf.

Frontend Experience

The interface presents the planning workflow as a conversation linked to a structured workspace. Customers can answer with typed controls, pause and resume, review the assembled brief, and amend individual answers before they authorize planning. The results view combines an editable timeline, map context, cost and payment guidance, and provenance details.

Core Interactions

- Conversational onboarding with accessible controls for dates, ranges, choices, rankings, and free text.
- Review and amend flow that shows exactly what the planner will use before confirmation.
- Editable itinerary with drag-and-drop and keyboard parity, local place search, deterministic recomputation, and scoped prose refresh.
- Per-item payment guidance, section-level freshness indicators, and trust badges that expose evidence status instead of hiding it.
- A destination-specific Japan visual system influenced by philatelic design while retaining semantic tokens, responsive behavior, and accessibility.

Frontend Financial Rule

The frontend never calculates a reward, fee, discount, or points value. It renders values from versioned backend contracts. This prevents browser logic from drifting from the audited financial kernel and keeps every displayed amount traceable to one computation.

Reliability and Quality Evidence

Backend tests	Typed Python files	Frontend tests	Browser tests
1,709 passed	**177 clean**	**163 passed**	**458 passed**

- Strict mypy validation covers 177 Python source files in the integrated acceptance scope.
- Ruff, TypeScript, accessibility tests, package-boundary checks, golden financial scenarios, and deterministic replay protect key invariants.
- OpenAPI regeneration is byte-stable, preventing unnoticed contract drift between FastAPI and the generated frontend client.
- The acceptance gate checks goldens, contracts, mirrored agent briefs, and a clean repository tree on the exact committed revision.

Resume Entry

SUGGESTED PROJECT HEADING

TripWise | Agentic AI Travel and Rewards Optimization Platform

Python, FastAPI, Pydantic, SQLAlchemy, SQLite, Next.js, React, TypeScript, MCP, OpenAPI, Pytest, Vitest, Playwright

Recommended Resume Bullets

- Architected a full-stack agentic travel planner that converts multi-turn customer interviews into confirmed Trip Briefs, personalized itineraries, and explainable card and loyalty-point strategies.
- Designed bounded AI workflows for intake, adaptive questioning, planning, critique, explanation, and post-plan edits using typed contracts, call budgets, deterministic fallbacks, and mandatory human approval gates.
- Built a deterministic Python optimization kernel for card rewards, spending caps, offer stacking, fees, cash-versus-points comparisons, and loyalty-transfer paths using integer monetary arithmetic and golden regression tests.
- Integrated read-only travel and weather evidence through application-owned MCP adapters with tool allowlists, schema validation, provenance, response bounds, circuit breakers, kill switches, and offline fixture replay.
- Implemented authenticated, resumable planning with Argon2id credentials, hashed server-side sessions, optimistic concurrency, idempotent events, immutable trip revisions, and privacy export and deletion.
- Delivered an accessible Next.js interface with typed interview controls, review and amend flows, editable itinerary timelines, keyboard parity, map context, payment guidance, trust badges, and generated OpenAPI clients.
- Established a high-confidence quality gate spanning 1,709 backend tests, 163 frontend unit tests, 458 browser tests, strict typing across 177 Python files, stable OpenAPI generation, and deterministic financial goldens.

Compact Four Bullet Version

- Built an agentic travel-planning platform that turns conversational preferences into confirmed, personalized itineraries and card and points recommendations.
- Separated LLM planning from deterministic financial computation, implementing exact reward, fee, offer, and loyalty-transfer logic with full provenance.
- Integrated travel, weather, place, FX, and airport evidence through bounded read-only MCP and data-gateway adapters with offline replay and fail-safe controls.
- Shipped a typed FastAPI and Next.js stack validated by 2,300-plus backend, frontend, and browser checks, strict typing, accessibility tests, and byte-stable API generation.

Elevator Pitches

Fifteen Second Version

I built TripWise, an agentic travel planner that learns how a customer likes to travel, creates a personalized itinerary, and deterministically calculates the best way to pay using their credit cards and loyalty points. AI handles interpretation and planning, while audited Python services handle every financial calculation.

Thirty Second Version

TripWise is a full-stack travel and rewards optimizer. It interviews a traveler, builds a reusable preference profile, gathers normalized flight, hotel, place, and weather evidence through controlled integrations, and creates a reviewable itinerary. A deterministic financial kernel then compares cash, card rewards, offers, and loyalty transfers. I designed the AI as bounded typed workflows with human confirmation, so it can act agentically without inventing prices or making irreversible decisions.

Sixty Second Version

I built TripWise to solve the gap between travel inspiration and financially sound trip planning. The product begins with a short conversational interview that learns a customer's pace, interests, comfort preferences, budget, credit cards, and points balances. A bounded set of AI workflows converts that into a confirmed Trip Brief, retrieves normalized evidence through read-only MCP and data-gateway adapters, proposes an itinerary, critiques it, and explains the result. The important architecture decision is that the LLM never performs financial math. A deterministic Python kernel calculates prices, rewards, caps, offers, fees, and transfer paths using integer arithmetic and produces a traceable artifact. The FastAPI and Next.js application supports authenticated resumable sessions, editable itineraries, provenance badges, and human approval before durable changes. The integrated quality gate covers more than 2,300 backend, frontend, and browser checks.

One Sentence Differentiator

TripWise connects personal travel taste with exact card and points optimization while keeping live evidence, model judgment, financial computation, and human approval visibly separate.

Best Framing by Audience

Audience	Emphasis
Software engineer	Lead with typed boundaries, deterministic computation, concurrency, idempotency, API contracts, and testing.
AI or ML engineer	Lead with bounded agents, tool controls, replay evaluation, grounded explanations, and deterministic fallbacks.
Product engineer	Lead with preference learning, review and amend UX, resumability, accessibility, and trust presentation.
Fintech or rewards team	Lead with exact arithmetic, card caps, offer ordering, transfer paths, provenance, and approval workflows.

Interview Talking Points

What Was the Hardest Problem

The hardest problem was combining probabilistic planning with exact financial decisions. Travel intent is naturally conversational and incomplete, but reward calculations must be deterministic. I solved that by using typed artifacts as the boundary between AI workflows and the financial kernel. Models can propose and explain; only deterministic services calculate and persist trusted financial outcomes.

How Did You Prevent Hallucinations

I prevented unsupported outputs at several layers. Provider data is normalized and labeled with provenance. Agent inputs and outputs use schemas. Tool access is allowlisted and bounded. The critic checks evidence and preference fit. Financial prose can only copy values from computed artifacts. When evidence is stale, missing, or malformed, the system returns a typed unavailable state or a verification warning instead of inventing a value.

Why Use MCP

MCP gave the project a consistent protocol for external tools, but the application still owns every security and product decision. Each adapter pins the endpoint and server version, exposes only approved read-only tools, validates arguments and responses, enforces call and payload limits, and converts results into provider-neutral contracts. That design makes providers swappable and keeps the core isolated from network behavior.

How Did You Handle Concurrency

State-changing repositories use optimistic concurrency and compare-and-swap versions. If two reviewers decide the same proposal from the same expected version, one transaction succeeds and the other receives a typed stale or already-decided conflict. Idempotency fingerprints let exact retries return the original result while rejecting a different request that reuses the same client event id.

What Would You Scale Next

I would move shared call budgets and coordination state from process-local storage to a durable distributed store, add more licensed provider options behind the existing gateway contracts, and benchmark model latency and retrieval quality under realistic multi-user load. The architecture already isolates those substitutions from the financial kernel and frontend contracts.

Responsible Project Positioning

Describe TripWise as a completed portfolio-grade decision-support platform. The strongest claims concern architecture, safety, correctness, interaction design, and verified test coverage. Avoid language that implies a commercial travel agency, guaranteed inventory, or autonomous financial execution.

Position	Language
Safe claim	The platform discovers and normalizes travel evidence, creates plans, and recommends payment strategies.
Safe claim	The system supports read-only provider integrations and clearly labels cached, fixture, estimated, and live evidence.
Safe claim	Users approve durable profile changes and verify availability before points transfers or purchases.
Avoid	TripWise autonomously books travel, executes points transfers, or guarantees prices and availability.
Avoid	TripWise is a production travel agency or has commercial rights to every provider's inventory.
Avoid	The AI calculates financial outcomes or replaces final verification by the traveler.

Final Resume Summary

Built TripWise, a full-stack agentic travel decision platform that combines conversational preference learning, normalized travel evidence, deterministic credit card and loyalty optimization, resumable planning, and human approval. Designed typed AI workflows and read-only MCP adapters with strict tool controls, provenance, replay testing, and fail-safe fallbacks. Delivered a FastAPI and Next.js application with exact financial arithmetic and more than 2,300 automated backend, frontend, and browser checks.

Keyword Bank

Agentic AI, LLM orchestration, Model Context Protocol, deterministic optimization, human in the loop, typed contracts, provenance, idempotency, optimistic concurrency, FastAPI, Pydantic, SQLAlchemy, Next.js, React, TypeScript, OpenAPI, accessibility, replay testing, circuit breakers, privacy, financial systems, travel technology.

Portfolio Note

For a public portfolio, pair this narrative with one architecture diagram, a short interaction video, two screenshots of the confirmed Trip Brief and results workspace, and one concrete calculation trace. Those artifacts make the separation between AI judgment and deterministic computation immediately visible.